MILO

APP FLOW & INTERACTION SPECIFICATION

Navigation architecture • User journeys • Screen states • Button-by-button behaviour

Version: 1.0 **Date:** 02 November 2026
Platform: Windows desktop, first release
Product: MILO — floating desktop island and personal productivity companion

Purpose

This document defines how a person moves through MILO, how the interface responds to every meaningful interaction, and what the application must do next. It is written as a build-facing companion to the Product Requirements Document, Technical Requirements Document, UI/UX Design Debrief, Implementation Plan, and Backend Data & Database Architecture Specification.

Implementation rule: every visible control must have a defined action, a defined resulting state, and a defined failure or cancellation path. No decorative button may look interactive.

Prepared as the interaction contract for implementation with Codex.

Document control and how to use this specification

Item	Definition
Primary audience	Codex, frontend and Rust/backend implementers, QA, product/design review.
Authority	Defines user-facing flow and interaction outcomes. The technical requirements document remains authoritative for implementation constraints and platform APIs.
Scope	First-run experience, collapsed/expanded shell, navigation, system deck, productivity, files, music, avatar, settings, notifications, tray, lifecycle, errors and recovery.
Out of scope	Pixel-perfect visual design tokens and native API implementation detail already specified in companion documents.
Normative language	MUST = required; SHOULD = expected unless documented reason; MAY = optional.
Versioning	Any change to a button outcome, route, confirmation, or persisted state must update this document and relevant tests.

How Codex should apply it

- Read all MILO documents before implementing a feature. Resolve conflicts by preserving the product intent and raising a decision rather than silently inventing behaviour.
- Implement navigation and state transitions as explicit, testable actions. Do not bury business logic inside presentation components.
- Use the button catalogue as a control inventory. Each control ID should map to a component, handler, backend command where applicable, and test.
- Build the shell and navigation before feature screens; implement one vertical slice at a time.
- Do not interpret a button as permission to perform an irreversible action without the confirmation rules defined here.

1. Product experience model

MILO is a persistent, lightweight desktop companion, not a conventional full-screen application. Its default experience is a compact island attached to the top-centre edge of the primary display. It expands downward when the user deliberately interacts with it. The expanded deck is the primary workspace; the Windows system tray is the secondary access point.

1.1 Experience goals

- Fast access: common actions should take one or two deliberate interactions from the collapsed state.
- Continuity: opening a section must not reset timers, lose drafts, or discard staged files.
- Low interruption: the island should not steal focus unexpectedly, cover important work, or expand because the pointer merely passes nearby for a moment.
- Predictability: the same action produces the same result; controls communicate whether they are toggles, commands, navigation, or destructive actions.
- Local-first: core interactions work without a MILO account or network connection.
- Recoverability: closing the panel is not equivalent to quitting MILO; app exit is an explicit lifecycle action.

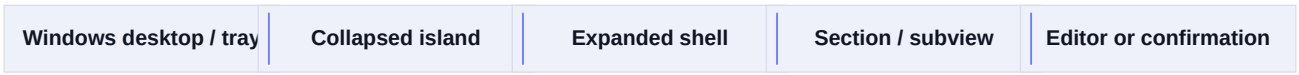
1.2 Core state model

State	Meaning	Entry	Exit
BOOTING	Process starts; configuration, database and native integrations initialise.	Windows launch, startup, tray reopen.	READY or DEGRADED/ERROR.
COLLAPSED	Island is compact and attached to screen edge.	Launch complete; user collapses; inactivity policy.	EXPANDING.
EXPANDING	Panel animates to expanded geometry.	Hover intent confirmed, click, hotkey, tray Open.	EXPANDED or COLLAPSED if interrupted.
EXPANDED	Deck is interactive; selected section visible.	Expansion completes.	COLLAPSING, SETTINGS overlay, modal, or route change.
COLLAPSING	Panel animates back to compact state.	User closes/collapses; outside-click policy; Escape where appropriate.	COLLAPSED.
MODAL	A focused decision/input dialog is open above the deck.	Destructive action, permission rationale, editor flow.	Prior state with result applied or cancelled.
DEGRADED	App remains usable but one or more integrations unavailable.	Native capability failure, permission denial, device unsupported.	READY after recovery/retry, or remain degraded.
EXITING	App is shutting down and flushing state.	Explicit Quit command or OS shutdown.	Process terminated.

Important distinction: collapsing hides the workspace but keeps MILO running. Quitting terminates the process. The UI must never label a collapse action as “Exit” or “Quit.”

2. Global navigation architecture

2.1 Navigation hierarchy



The expanded shell uses a stable header, a primary navigation rail or section selector, a content viewport, and contextual actions. The navigation structure must remain consistent while the content changes.

Level	Destination	Purpose	Navigation rule
Shell	Overview	Default landing view; high-value summary and quick actions.	Default route on normal open unless a notification deep-links to a specific item.
Primary	System	Time/date and available system controls.	One persistent section.
Primary	Productivity	Tasks, reminders, timers, alarms.	Subsections inside Productivity.
Primary	Files	Temporary file staging/dropzone.	Never silently move or delete source files.
Primary	Music	Now playing, transport controls, lyrics availability.	Music integration may be unavailable without disabling other sections.
Primary	Settings	Preferences, permissions, startup, appearance, shortcuts, privacy, about.	Opens as a dedicated view or overlay; preserves previous section.
Contextual	Search / quick actions	Find and invoke an action or item.	Results route to relevant item or execute an explicitly selected command.
Contextual	Notifications	Recent MILO alerts and actionable items.	Selecting an item deep-links to its source.
Contextual	Avatar / companion	Companion presence and optional customisation.	Avatar is visual companion, not primary navigation.

2.2 Navigation behaviours

- Selecting a primary section changes the content viewport without creating a second app window.
- Selecting the already-active section is idempotent: it does not reset scroll position, forms, timers or playback.
- Switching sections preserves unsaved drafts in memory and persists durable entities at the correct save point.
- An active timer, reminder, alarm or staged file remains active/available when the user navigates elsewhere or collapses the island.
- Back returns to the previous view within the current navigation stack; it must not unexpectedly collapse MILO.
- Escape closes the topmost transient layer first (popover, menu, modal, then search). It only collapses the deck when no transient layer is open and the current view allows it.
- Deep links from notifications open MILO, expand it, select the correct section, and focus the referenced item.

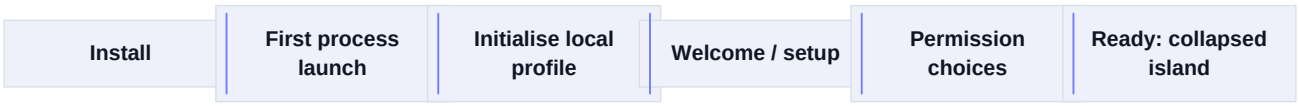
2.3 Navigation stack and state retention

User action	Expected result
Overview → Tasks	Tasks view opens; overview scroll state is retained.
Tasks → edit task → Cancel	Return to Tasks; task remains unchanged.
Tasks → edit task → Save	Persist task; return to Tasks with updated row and success feedback.
Tasks → System → Tasks	Return to Tasks at prior scroll position; no duplicate route.
Expanded → Collapse → Reopen	Restore last active section unless a notification deep-link overrides it.
Settings → Back	Return to exact prior section and subview.
Notification → task detail	Open the task in its parent section; back returns to notification context.

3. End-to-end application lifecycle

The app lifecycle describes how MILO starts, becomes available, stays resident, recovers, and exits. Treat these as explicit states rather than incidental window visibility.

3.1 First installation and first launch



Step	What happens	User-facing outcome
1. Install	Installer lays down signed app files and required runtime components.	MILO appears in Windows app list and Start menu.
2. Launch	Single-instance guard checks for existing process.	If already running, bring existing instance forward; do not create a duplicate.
3. Initialise	Create/verify local app data directory, database, schema, default preferences, migration state.	Show brief loading state; never expose a blank broken panel.
4. Detect environment	Read display geometry, DPI, Windows version/capabilities, theme and available integrations.	Choose safe default placement and supported feature states.
5. Welcome	Explain what the floating island is and how to open/collapse it.	User can continue or defer optional setup.
6. Preferences	Offer startup preference, theme, hotkey and notification preferences as optional choices.	Do not block core use for optional settings.
7. Permissions	Request native permissions only when the user invokes a feature that needs them, with clear rationale.	Permission denied means feature remains unavailable with a route to Settings/help.
8. Ready	Start background scheduler/services, register tray icon and configured shortcuts, show collapsed island.	User reaches the default state.

3.2 Returning launch

- Load saved preferences and last active section, unless a deep-link action has priority.
- Restore only safe UI state. Do not restore a modal, half-completed destructive confirmation, or stale transient menu.
- Reconcile overdue reminders/alarms using stored schedule and occurrence state; avoid duplicate notifications.
- Restore staged-file metadata only. Verify that referenced paths still exist before presenting a file as available.
- Reconnect to supported media/system integrations without blocking shell readiness.
- Start collapsed by default unless the user has explicitly chosen a different launch preference.

3.3 Running in background

- Closing the expanded panel collapses the island; it does not terminate timers, reminders, alarms, media monitoring or file staging.
- Tray menu remains available while MILO is resident.
- Background services should be event-driven or low-frequency; avoid busy polling.
- Notifications may appear according to user preferences and Windows notification capability.
- On lock, sleep, resume, display change or user-session transition, refresh relevant integrations and schedule calculations.

3.4 Quit

Action	Required behaviour
Tray → Quit	Ask for confirmation only if there is unsaved work that cannot be safely persisted; otherwise quit directly or use a clear confirmation preference.
Settings → Quit MILO	Same behaviour as tray Quit.
Window close affordance	Collapse by default, not quit. Tooltip and accessible name must say “Collapse MILO” or “Hide MILO”.
Windows shutdown/sign-out	Flush durable writes, stop timers/services cleanly, and avoid blocking OS shutdown.
Re-launch after quit	Start a fresh process, restore durable state, reconcile schedules, and show collapsed.

4. Collapsed island and expansion flow

The island is the signature entry point. It must feel immediate and intentional, while respecting accidental pointer movement and full-screen work.

4.1 Collapsed state anatomy

- Compact island handle centred on the top edge; displays a restrained status cue such as time, active timer, or notification indicator.
- Avatar is not displayed in collapsed state unless a specific compact indicator is approved by design.
- Click target is large enough for reliable mouse use and accessible to keyboard/screen-reader navigation.
- Hover can begin an intent timer, but expansion must not trigger on a momentary pointer pass.
- Collapsed island must remain above normal desktop content only as specified by the window z-order policy; it must not trap input.

4.2 Expansion triggers

Trigger	Sequence	Result
Pointer hover	Pointer enters target → intent delay → verify pointer still present → animate expansion.	Expanded Overview.
Click	Pointer click on island handle.	Expand immediately; focus first meaningful control.
Configured global hotkey	Shortcut event received → check app state → expand or collapse.	Toggle shell; do not steal focus if shortcut is disabled.
Tray → Open MILO	Tray action → restore window → expand.	Expanded Overview or deep-linked destination.
Notification action	OS notification click/action → restore process → expand → route to item.	Relevant section and item focused.

4.3 Collapse triggers

- Collapse button in header.
- Escape when no modal, menu, popover or search overlay is active.
- Click outside only when the click is not targeting another application control that would be disrupted; do not consume the outside click.
- Optional inactivity collapse after a configurable period, but never while the user is typing, dragging files, using a modal, or interacting with a control.
- Global hotkey toggle.
- Collapse animation completes to compact island; app process and background services remain active.

4.4 Interruption and edge cases

Scenario	Required behaviour
User moves pointer away during hover delay	Cancel expansion.
User moves pointer away during expansion	Complete or reverse animation smoothly; never leave a half-sized hit target.
User clicks a control while expansion is animating	Queue/route the click only if hit target is stable; otherwise finish expansion first.
User starts dragging a file	Suspend auto-collapse until drag ends or cancels.
Full-screen application active	Respect full-screen suppression preference; hotkey/tray can still open if configured.
Display topology changes	Recalculate island anchor and safe bounds; preserve current route and data.
DPI changes	Re-layout using logical dimensions and updated scale factor; do not resize unpredictably.

5. Expanded shell: global controls and interaction contract

5.1 Persistent shell elements

Control	Type	Action / next state
MILO mark / home	Navigation	Select Overview; preserve current route for Back if implemented as history.
Section navigation item	Navigation	Load selected section in content viewport.
Collapse chevron	Window command	Animate to collapsed state; app remains running.
Search icon/button	Overlay command	Open search overlay; focus search field; Esc closes overlay.
Notification bell	Overlay/navigation	Open notification centre; unread indicator updates.
Avatar	Companion/customisation	Open companion panel or avatar settings; must not be the only route to a feature.
Settings gear	Navigation	Open Settings; preserve previous route.
More/context menu	Menu	Show contextual actions; clicking outside or Esc dismisses.
Back	Navigation	Pop one view from navigation stack; disabled when no back target.

5.2 Universal button states

State	Visual / interaction contract	Behaviour
Default	Clearly interactive; consistent icon/label and hit target.	Accept one activation.
Hover	Subtle emphasis, no layout jump.	No action until activation.
Pressed	Immediate tactile visual feedback.	Prevent duplicate invocation.
Focused	Visible keyboard focus ring.	Enter/Space activates as appropriate.
Loading	Spinner/progress or busy label.	Disable duplicate submission; retain context.
Success	Short confirmation or updated content.	Return to stable state; do not leave spinner.
Disabled	Muted but legible; reason available where useful.	No operation; tooltip/help explains why.
Error	Inline or toast feedback tied to action.	Preserve input and allow retry/correction.
Destructive	Warning treatment and explicit label.	Confirmation step for irreversible/high-impact action.

All buttons must be operable by keyboard. Icon-only buttons require an accessible name and a tooltip. A click handler must not silently do nothing.

6. Overview: default landing view

Overview is the at-a-glance home. It should summarise useful state and provide direct entry points without becoming a crowded dashboard.

6.1 Content and user path

- Welcome/time/date region with contextual greeting kept secondary to useful information.
- Compact system status summary: battery/power where available, connectivity status where supported, and active media.
- Productivity summary: due/overdue tasks, next reminder, active timers and next alarm.
- Files summary: number of staged items and clear action to open Files.
- Quick actions: add task, start timer, add reminder, open Files, open Music where relevant.
- Avatar appears in expanded state as a restrained companion element; it must not cover content or interrupt tasks.

6.2 Overview button inventory

Control ID / button	On activation	Next state / result	Failure / special case
OV-01 Add task	Open task creation form with focus in title.	Save creates task; Cancel returns to Overview unchanged.	Validation shown inline; preserve typed values.
OV-02 Start timer	Open quick timer preset/input.	Start creates active timer and shows running state.	Invalid duration blocks start and explains valid range.
OV-03 Add reminder	Open reminder editor.	Save persists reminder; return to Overview with next reminder refreshed.	If scheduling fails, preserve draft and show retry.
OV-04 View all tasks	Navigate to Productivity → Tasks.	Tasks list with relevant filter if clicked from due summary.	Empty state offers Create task.
OV-05 View reminders	Navigate to Productivity → Reminders.	Reminder list, sorted by upcoming/overdue.	Empty state offers Add reminder.
OV-06 View timers	Navigate to Productivity → Timers.	Timer list / active timer panel.	If none, show presets and Create timer.
OV-07 Open Files	Navigate to Files.	Dropzone and staged-file list.	If folder unavailable, show recoverable error.
OV-08 Open Music	Navigate to Music.	Now Playing / idle state.	If no supported session, explain and show unavailable state.
OV-09 Play/Pause	Send transport command to active media session.	Playback state and metadata refresh.	If no session, disabled with explanation.
OV-10 Dismiss card	Dismiss only that overview card for current preference/session as defined.	Card removed; underlying entity unchanged.	Do not delete task/reminder/file.
OV-11 Refresh status	Request fresh system snapshot.	Status cards update; timestamp refreshed.	Keep last-known values with stale indicator if refresh fails.

6.3 Overview card rules

- Cards are summaries, not separate data stores. Clicking a card opens the canonical feature view.
- Dismiss removes a card from the overview only; it never deletes the underlying task, reminder, timer or staged file.
- Empty cards should offer a direct creation action.
- Do not show fake data or sample values after onboarding is complete.
- Show unavailable integration states explicitly rather than rendering misleading zero values.

7. System deck: system status and controls

System controls are potentially sensitive because they affect the Windows device. Every control must distinguish MILO preference changes from native system changes.

7.1 System view structure

- Time and date, formatted according to Windows locale and user preferences.
- Battery and power status, with charging/low-battery cues only when the OS reports them.
- Volume and brightness controls where supported; Bluetooth and Wi-Fi/network status/actions only where reliable APIs and permissions allow.
- Energy saver / power mode entry where available, with capability and OS-version checks.
- Unavailable controls remain visible only if the design intentionally communicates capability; otherwise hide them. Never present a fake working toggle.

7.2 System button/control catalogue

Control ID	Control	Activation and next step	Error / guard
SYS-01	Volume slider	Changing value requests native volume level; show updated percentage and mute state.	If endpoint unavailable, revert to last confirmed value and show error.
SYS-02	Mute toggle	Toggle mute; query OS for confirmed state; update icon/label.	Do not assume success before native response.
SYS-03	Brightness slider	Set selected display brightness if supported; reflect confirmed level.	If unsupported or denied, explain; do not simulate.
SYS-04	Brightness display selector	Choose target display when multiple displays exist.	Changing target does not alter brightness until slider changes.
SYS-05	Wi-Fi/network status	Open network details or Windows network settings if direct control unavailable.	Do not claim connected/disconnected without OS evidence.
SYS-06	Bluetooth status/action	Open supported Bluetooth control or Windows Settings page.	If native toggle cannot be implemented safely, provide settings deep link.
SYS-07	Energy saver toggle	Request OS power-mode action if supported; confirm resulting state.	Show unsupported/permission state and link to Windows settings.
SYS-08	Battery details	Expand battery/power details.	If no battery, show “Desktop power” state, not an error.
SYS-09	Open Windows Settings	Launch relevant OS settings page.	If URI launch fails, show manual route/help.
SYS-10	Refresh system status	Re-query native status snapshot.	Retain last confirmed reading with stale marker on failure.

7.3 Toggle contract

A toggle is not a decorative switch. It must represent a confirmed state, not merely the user's click. On activation: disable rapid repeat; send request; await native response; query or consume confirmed state; update UI; announce result. On failure, restore the previous confirmed state and explain why.

For controls Windows does not expose reliably through a supported API, MILO must open the relevant Windows Settings surface or present a clearly labelled limitation. Do not fake native control.

8. Productivity: shared navigation and entity patterns

Productivity contains Tasks, Reminders, Timers and Alarms. These features share consistent list, create, edit, delete, search/filter and empty-state patterns, but their scheduling semantics differ.

8.1 Shared list pattern

Element	Behaviour
Create button	Open a focused creation form; save creates one entity; cancel returns without persistence.
Search	Filter visible entities; clearing restores list; search must not modify records.
Filter chips	Apply view-only filters; active filter is visually obvious and resettable.
Sort control	Change ordering only; persist preference if intended.
Row click	Open detail or editor; clicking checkbox/inline control must not open row detail.
Overflow menu	Offer valid actions for the entity: edit, duplicate where appropriate, archive/delete, mark complete.
Empty state	Explain what belongs here and show one primary creation action.
Bulk actions	Only appear after selection; destructive bulk operations require clear count and confirmation.

8.2 Shared editor rules

- Required fields are visibly marked; validation appears beside the relevant field.
- Save validates locally first, then persists; show loading state while the save is pending.
- Successful save closes the editor and updates the originating list/summary.
- Cancel closes the editor without changing the existing entity. If there are unsaved changes, ask whether to discard or continue editing.
- Delete is not placed beside Save as an equally prominent primary action; it is in an overflow/destructive area.
- After a save error, keep all user-entered values intact.

9. Tasks: complete user journey and button map

9.1 Task journey

Productivity	Tasks list	Create / edit task	Validate	Save	Updated list / detail

9.2 Task fields and states

- Title (required), optional description, due date/time, priority, status, tags, checklist items and optional reminder linkage.
- Statuses: Open, In progress, Completed, and optionally Archived if supported by the database/product model.
- Overdue is derived from due date/time and completion state; it is not a separate manually set status.
- Checklist completion is independent of task completion unless a documented rule is chosen.
- Completed tasks remain available in completed/history filters; they are not silently deleted.

9.3 Task button catalogue

Control ID	Button / control	On activation	Next / result
TSK-01	New task	Open blank task editor; title focused.	Save → create task; Cancel → task list.
TSK-02	Task row	Open task detail/editor.	Back → same list position.
TSK-03	Complete checkbox	Toggle completion and persist.	Row updates; completion timestamp stored; undo toast where feasible.
TSK-04	Edit	Open editor prefilled from canonical record.	Save updates; Cancel discards draft.
TSK-05	Delete	Open confirmation with task title.	Confirm deletes/soft-deletes according to retention rules; Cancel leaves task.
TSK-06	Duplicate	Create a new draft copied from source without identity/history.	Save creates distinct task; no reminder occurrence copied unless explicitly selected.
TSK-07	Add checklist item	Append editable checklist row.	Enter/Save commits item; empty item is not persisted.
TSK-08	Remove checklist item	Remove from draft or ask confirmation if already persisted.	Save applies; Cancel retains item.
TSK-09	Add tag	Open tag picker/create flow.	Select tag applies to draft; Save persists relationship.
TSK-10	Clear filters	Reset filters/search to default list.	All applicable tasks visible.
TSK-11	Sort	Open sort options.	Selection reorders list; does not alter task data.
TSK-12	Mark all complete	Confirmation with affected count.	Confirm updates selected scope; Cancel no changes.
TSK-13	Archive	Move task out of active list if archive is supported.	Undo available briefly where feasible.
TSK-14	Restore	Return archived task to active list.	Task reappears in relevant sort position.
TSK-15	Save task	Validate and persist.	Close editor; refresh list and overview summary.
TSK-16	Cancel	Close editor.	If dirty, show discard confirmation; otherwise return immediately.

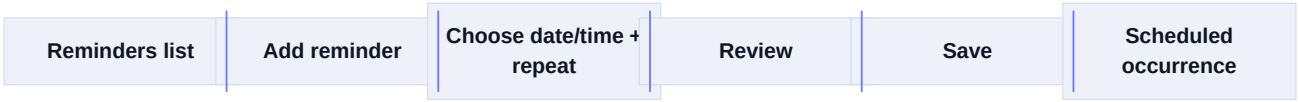
9.4 Task edge cases

- Duplicate titles are allowed unless product explicitly chooses uniqueness; identity is the task ID.

- Deleting a task with linked reminder data must follow database relationship rules and explain whether linked reminder is retained, detached or deleted.
- Changing due date/time recalculates overdue and reminder scheduling; it must not create duplicate occurrences.
- Completing a task with a future reminder should either cancel the linked reminder or ask the user according to the chosen product rule. Codex must not invent a silent policy.
- Task list updates must remain correct after app restart and when modified by notification actions.

10. Reminders: scheduling and interaction flow

10.1 Reminder journey



10.2 Reminder button catalogue

Control ID	Button / control	On activation	Next / result
REM-01	Add reminder	Open reminder editor.	Save → schedule; Cancel → list.
REM-02	Reminder row	Open reminder details/editor.	Back → prior list position.
REM-03	Date picker	Open calendar selection.	Selected date appears in editor; no schedule until Save.
REM-04	Time picker	Open time selection.	Selected time appears; invalid/past handling shown.
REM-05	Repeat selector	Choose none/daily/weekly/monthly/custom if supported.	Show recurrence summary and next occurrence preview.
REM-06	Save reminder	Validate date/time/repeat and persist.	Create/update schedule; confirmation includes next fire time.
REM-07	Snooze	Choose configured snooze interval or preset.	Create snoozed occurrence; preserve original recurrence.
REM-08	Dismiss	Dismiss current occurrence.	Mark occurrence handled; future recurrence remains.
REM-09	Mark complete	Mark linked reminder/task occurrence complete if model supports it.	Update occurrence and linked task status per defined relationship.
REM-10	Delete reminder	Confirmation names reminder and explains recurrence removal.	Confirm removes future schedule and related pending occurrences per retention rules.
REM-11	Enable/disable	Pause or resume reminder schedule.	Confirm persisted enabled state; next occurrence recalculated.
REM-12	Open source task	Navigate to linked task.	Task opens in Productivity → Tasks.
REM-13	Cancel editor	Discard draft if dirty; otherwise close.	Return to list without schedule changes.

10.3 Time and recurrence rules

- Store timestamps with a clear timezone strategy; display in local Windows time.
- Handle daylight-saving transitions explicitly: nonexistent local times should be adjusted or confirmed; repeated local times must not double-fire.
- At app start/resume, reconcile due occurrences and deliver at most one notification per occurrence ID.
- Snoozing creates a new occurrence linked to the original; it must not mutate the recurrence rule.
- Disabling a reminder prevents future firing but preserves the reminder record and its configuration.
- Notification permission denial does not erase reminders; show them in MILO and offer Windows notification settings.

11. Timers: creation, running, pause and completion

11.1 Timer journey



11.2 Timer button catalogue

Control ID	Button / control	On activation	Next / result
TMR-01	New timer	Open duration editor.	Save creates timer definition; Start begins run.
TMR-02	Preset duration	Populate duration from preset.	User may start immediately or adjust.
TMR-03	Start	Create timer run with monotonic timing strategy.	Running view; countdown updates without drift.
TMR-04	Pause	Record paused state and remaining duration.	Resume button becomes primary.
TMR-05	Resume	Resume from stored remaining duration.	Running state restored.
TMR-06	Reset	Confirmation if timer has progressed materially; reset remaining time.	Idle or ready state.
TMR-07	Stop	End active run.	Return to idle; history retained if configured.
TMR-08	Add time	Add selected interval to remaining duration.	Countdown recalculates; show updated total.
TMR-09	Complete alert: Dismiss	Acknowledge alert.	Timer run marked acknowledged/completed.
TMR-10	Complete alert: Restart	Start a new run using same duration.	New run ID; prior run remains historical.
TMR-11	Delete timer	Confirm deletion of timer definition.	Active run must be stopped/handled explicitly before delete.
TMR-12	Save timer	Validate duration/name and persist.	Return to timer list with new timer.
TMR-13	Cancel	Discard editor draft.	Return to prior timer view.

11.3 Timer correctness

- Countdown display must be derived from a reliable time source, not repeated subtraction on UI ticks.
- Timer continues when the island collapses and while the app is in the background.
- On sleep/resume, recompute remaining time against intended elapsed-time semantics.
- Only one completion notification per timer run may be emitted.
- System clock changes must not cause negative countdowns or duplicate completion events.

12. Alarms: recurring schedule and response

12.1 Alarm journey

Alarms list	Create alarm	Set time / days	Enable	Alarm fires	Snooze or dismiss

12.2 Alarm button catalogue

Control ID	Button / control	On activation	Next / result
ALM-01	Add alarm	Open alarm editor.	Save creates disabled or enabled alarm according to explicit default.
ALM-02	Time selector	Choose local clock time.	Preview next fire time.
ALM-03	Repeat days	Toggle weekdays individually.	Summary updates (e.g. weekdays, weekends, custom).
ALM-04	Enable toggle	Enable/disable alarm schedule.	Persist state; show next occurrence if enabled.
ALM-05	Save alarm	Validate time/day selection and persist.	Return to list; confirmation of schedule.
ALM-06	Edit alarm	Open prefilled editor.	Save updates; Cancel preserves prior schedule.
ALM-07	Delete alarm	Confirm deletion.	Confirm removes schedule; Cancel no changes.
ALM-08	Snooze	Schedule next ring after configured interval.	Current ring dismissed; alarm remains scheduled.
ALM-09	Dismiss	Stop current ring.	Record acknowledgement; next recurrence remains.
ALM-10	Open alarm settings	Open notification/sound options.	Save persists preference.
ALM-11	Cancel editor	Discard unsaved changes if any.	Return to list.

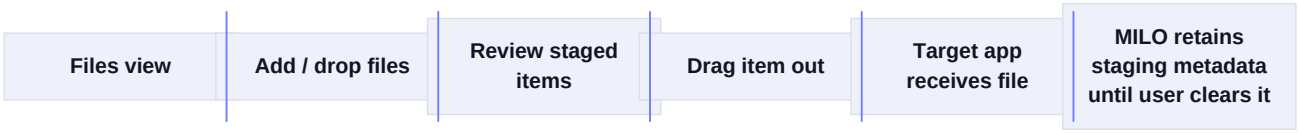
12.3 Alarm delivery constraints

- Clearly disclose if Windows sleep state, app termination, notification policy or platform limitations can prevent reliable alarm delivery.
- Do not claim guaranteed wake-from-sleep behaviour unless a tested Windows mechanism supports it and the product explicitly implements it.
- Provide a test alarm action so users can verify sound/notification configuration.
- Alarm sound must respect user mute/volume preference and accessibility requirements; avoid sudden excessive volume.
- An alarm firing must be actionable from the notification and from the expanded MILO view.

13. Files: temporary staging and drag-out workflow

Files is a holding area for temporary staging. It is not a file manager, sync folder, or silent file-move utility.

13.1 Main journey



13.2 Files button and gesture catalogue

Control ID	Button / gesture	On activation	Next / result
FIL-01	Add files	Open native file picker.	Selected paths are staged; source files remain where they are.
FIL-02	Drop files into zone	Receive native file-drop event and validate paths.	Create staged-file records; show names, size and status.
FIL-03	Staged file row	Open file details/preview metadata.	No source file mutation.
FIL-04	Drag staged item out	Begin native drag-out operation using source path/data object.	Target app receives file; MILO records drag outcome where observable.
FIL-05	Open file	Launch default Windows application for file.	External app opens; MILO remains available.
FIL-06	Reveal in Explorer	Open containing folder and select file where supported.	Explorer foregrounds; source unchanged.
FIL-07	Copy path	Copy canonical path to clipboard.	Show copied feedback.
FIL-08	Remove from staging	Remove staging record only, unless user separately chooses file deletion.	Confirm if user may confuse removal with deleting source.
FIL-09	Clear all staged	Show count and explicit “Remove from MILO staging” wording.	Confirm removes records; never deletes originals.
FIL-10	Retry unavailable file	Recheck original path and permissions.	If present, restore available status; otherwise explain.
FIL-11	Sort/filter	Reorder visible staging list.	No file mutation.
FIL-12	Privacy/help	Explain local metadata and retention.	Open help/policy view.

13.3 Drag-out state machine

State	Expected behaviour
IDLE	Staged row can be selected or dragged.
DRAG_ARMED	Pointer movement crosses threshold; prepare native drag source.
DRAGGING	OS drag cursor/target feedback is shown; suspend collapse and avoid intercepting target app.
DROPPED	Target accepts or OS reports completion; record outcome if known.
CANCELLED	User presses Esc or returns to source; keep staged item unchanged.
FAILED	Native drag operation cannot start; preserve staged item and show retry guidance.

Never interpret drag-out as a request to move or delete the source. The original path remains authoritative. Staging removal and source deletion are separate actions; source deletion is not part of the normal Files workflow.

13.4 File edge cases

- Path no longer exists: show Missing, retain metadata until user removes it, and offer retry/reveal only when valid.

- Access denied: show permission issue; do not copy file into MILO silently.
- Duplicate path: prevent confusing duplicate rows or label duplicate entries clearly.
- Large file: avoid loading full contents into memory merely to show metadata.
- Network/removable path unavailable: mark temporarily unavailable and recheck when requested.
- Target app rejects drag: user can use Open, Copy path or retry; do not mark transfer as successful without evidence.

14. Music: media session and lyrics journeys

14.1 Music view

- Now Playing area displays title, artist, album/artwork when exposed by a supported Windows media session.
- Transport controls reflect the active session's actual state.
- Lyrics are displayed only when a supported and permitted source is available; unavailable lyrics must not be fabricated.
- Music controls may be unavailable when no controllable session exists; the rest of MILO remains fully functional.

14.2 Music button catalogue

Control ID	Button / control	On activation	Next / result
MUS-01	Play/Pause	Send transport command to active media session.	Refresh confirmed playback state.
MUS-02	Previous track	Send previous command.	Metadata updates when session reports new item.
MUS-03	Next track	Send next command.	Metadata updates when session reports new item.
MUS-04	Seek bar	Seek to selected position if session supports it.	Show confirmed position; disable if unsupported.
MUS-05	Volume	Adjust MILO-controlled output only if supported and clearly labelled.	Do not confuse with system master volume.
MUS-06	Show lyrics	Open lyrics panel and query supported source.	Show lyrics, loading, unavailable or error state.
MUS-07	Hide lyrics	Close lyrics panel.	Return to Now Playing layout.
MUS-08	Favourite	Toggle favourite for supported track metadata.	Persist local favourite; show confirmation.
MUS-09	Open source app	Open or foreground source app when supported.	If unavailable, explain; do not launch guessed executable.
MUS-10	Retry media detection	Refresh session enumeration.	Update controls and metadata or show no-session state.

14.3 Media state transitions

Detected condition	UI response
No session	Idle illustration/text; transport controls disabled; retry guidance.
Session active, paused	Show metadata and Play action.
Session active, playing	Show metadata, Pause action, elapsed/seek if supported.
Session metadata partial	Display available fields; hide missing artwork/lyrics rather than invent.
Session lost	Keep last metadata briefly with disconnected/stale label; disable transport.
Lyrics source unavailable	Show “Lyrics unavailable for this track/source” and retain music controls.
Lyrics loading	Show skeleton/loading indicator; prevent repeated duplicate requests.

15. Avatar and companion interactions

The avatar is a companion layer, not a gatekeeper, chatbot requirement, or replacement for clear labels. It appears in the expanded deck and must never obscure system controls or critical information.

Control ID	Button / gesture	On activation	Next / result
AVA-01	Avatar tap/click	Open companion panel or approved expression menu.	Panel shows available, non-essential interactions.
AVA-02	Expression selector	Choose supported expression.	Preview immediately; persist only after confirm if customisation flow requires.
AVA-03	Animation/motion preference	Toggle reduced/subtle motion options.	Apply immediately; persist preference.
AVA-04	Hide companion	Disable avatar display.	Avatar disappears; all app features remain accessible.
AVA-05	Reset companion	Restore default avatar settings.	Confirm if customisation is lost; apply defaults.
AVA-06	Close companion panel	Dismiss panel.	Return to prior section without route loss.

- Avatar animation pauses or reduces when the app is backgrounded or system performance is constrained.
- Reduced motion preference is respected.
- Avatar does not speak, play sounds or create notifications unless explicitly enabled and designed.
- Avatar presence never blocks keyboard focus or screen-reader access to real controls.

16. Search, quick actions and global command flow

16.1 Search behaviour



Control ID	Button / control	On activation	Next / result
SEA-01	Search icon	Open search overlay and focus input.	Search-ready state.
SEA-02	Search field	Update local query.	Show grouped results; debounce without losing keystrokes.
SEA-03	Clear query	Empty search input.	Restore recent/suggested actions if available.
SEA-04	Search result: entity	Open canonical entity detail.	Back returns to search results and query.
SEA-05	Search result: command	Show action confirmation if impact warrants it.	Execute only after explicit selection; report result.
SEA-06	Close search	Close overlay.	Return to previous section and focus origin control.

16.2 Quick actions

- Quick actions are shortcuts to known MILO operations: create task, start timer, create reminder, open Files, open Music, open Settings, collapse/expand.
- Quick actions must use the same underlying handlers as their corresponding screen buttons; do not fork business logic.
- Actions that create or modify data open the same form/confirmation used in the canonical section.
- Global shortcuts are configurable and conflict-checked. A failed registration must be shown as unavailable, not silently ignored.

17. Notifications and notification centre

17.1 Notification lifecycle



Control ID	Button / action	On activation	Next / result
NOT-01	Notification bell	Open notification centre.	List recent MILO notifications; unread count updates.
NOT-02	Notification row	Open source entity.	Expand MILO and focus relevant item.
NOT-03	Mark read	Mark selected notification read.	Unread badge decrements.
NOT-04	Mark all read	Mark visible/all notifications read after scope is clear.	Unread badge clears.
NOT-05	Snooze action	Apply configured snooze to reminder/alarm occurrence.	New occurrence scheduled; original remains linked.
NOT-06	Dismiss action	Acknowledge current notification.	Notification removed from active list; source entity remains.
NOT-07	Clear history	Confirmation with scope.	Clear notification records only; never delete tasks/reminders.
NOT-08	Notification settings	Open Settings → Notifications.	Preserve notification-centre context for Back.

17.2 Delivery rules

- Create a stable notification/occurrence ID before attempting delivery so retries do not duplicate alerts.
- Store notification outcome: pending, delivered where known, dismissed, snoozed, failed or expired.
- Notification click must restore an existing process rather than spawning a second instance.
- Notification actions must validate that the referenced entity still exists and is actionable.
- If OS notifications are disabled, in-app notification centre still works and Settings explains how to re-enable OS delivery.

18. Settings: information architecture and every control

18.1 Settings sections

Section	Controls and purpose
General	Launch behaviour, close/collapse behaviour, default landing section, language/locale if supported.
Appearance	Dark/light/system theme, accent preference if exposed, transparency/reduced effects.
Island behaviour	Hover delay, auto-collapse, screen-edge placement, full-screen suppression.
System integrations	Status refresh, supported device controls, capability diagnostics.
Notifications	OS notification preference, sound, reminder/alarm delivery, quiet hours if supported.
Shortcuts	Global hotkey assignments, conflict detection, enable/disable.
Productivity	Default reminder snooze, timer presets, week start, time format.
Files & privacy	Staging retention, clear staging, metadata explanation, privacy controls.
Music	Media detection, lyrics availability/source preferences, metadata display.
Companion	Show/hide avatar, animation intensity, reduced motion.
Data	Backup/export, restore/import where supported, reset local data with confirmation.
About & updates	Version, update check, release notes, diagnostics, licenses, support.

18.2 Settings control catalogue

Control ID	Control	On activation	Next / result
SET-01	Launch at startup toggle	Request autostart registration/unregistration.	Reflect confirmed OS registration; show failure if blocked.
SET-02	Start collapsed toggle	Persist launch-state preference.	Applies next launch unless stated otherwise.
SET-03	Theme selector	Choose Dark / Light / System.	Apply immediately; preserve contrast.
SET-04	Accent selector	Choose supported accent.	Preview immediately; persist on selection.
SET-05	Reduced motion toggle	Disable/reduce non-essential animation.	Apply immediately across shell and avatar.
SET-06	Hover delay	Adjust expansion intent delay.	Preview after change; persist.
SET-07	Auto-collapse toggle/time	Enable and set inactivity duration.	Apply only when safe; never collapse during modal/drag/typing.
SET-08	Full-screen suppression	Suppress automatic expansion over full-screen apps.	Manual hotkey/tray remains available.
SET-09	Notification permission/help	Open OS notification settings or permission explanation.	Return to Settings after OS page closes.
SET-10	Test notification	Send a test notification.	Show success/failure; no real reminder created.
SET-11	Shortcut recorder	Capture requested key combination.	Validate conflict; save only if available.
SET-12	Reset shortcut	Restore default shortcut.	Confirm if user customised it.
SET-13	Timer preset editor	Add/edit/remove preset.	Validate duration; persist preset list.
SET-14	Default snooze selector	Choose default interval.	Used by future snooze actions.
SET-15	Clear staged files	Open staging-clear confirmation.	Remove staging metadata only.
SET-16	Export data	Choose export destination and scope.	Create export file; show result and path.
SET-17	Import/restore	Choose backup and preview changes.	Require confirmation; validate backup before applying.
SET-18	Reset data	Show destructive confirmation and consequences.	Require explicit confirm; backup recommendation; reset and restart/reinitialise.
SET-19	Check for updates	Query configured update source.	Show available/current/error; never install silently unless policy allows.
SET-20	View release notes	Open in-app notes or trusted URL.	Return to About.
SET-21	Open support/diagnostics	Show local diagnostic bundle options.	User chooses whether to export; do not transmit automatically.
SET-22	About / licenses	Show version and third-party notices.	No data mutation.
SET-23	Back	Return to previous route.	Preserve prior section and scroll state.

18.3 Settings save model

- Simple toggles apply immediately and show confirmed state.
- Multi-field forms use explicit Save/Cancel; unsaved changes are tracked.
- System-level preferences may require a permission or OS settings handoff.
- Reset, restore and data deletion are destructive and require clear scope, consequence and confirmation.
- Settings must distinguish “MILO preference” from “Windows setting” in labels and helper text.

19. Tray menu, global shortcuts and external entry points

19.1 Tray menu

Tray item	On activation	Next state
Open MILO	Restore existing window and expand.	Overview or pending deep-link.
Quick add task	Open task creation editor.	Save → create; Cancel → return to prior shell state.
Start timer	Open quick timer controls.	Start → running timer; Cancel → no timer.
Pause/resume timer	Toggle active timer state.	Tray label reflects confirmed state.
Notifications	Open notification centre.	Unread list.
Settings	Open Settings.	Prior route stored for Back.
Check for updates	Run update check.	Show result in MILO.
Quit	Run safe shutdown flow.	Process exits after durable writes.

19.2 Global shortcut contract

- Default shortcut is documented in onboarding and Settings; user can change it.
- Shortcut toggles collapsed/expanded state only when that action is not intercepted by a higher-priority modal interaction.
- Shortcut registration failure displays the conflicting/unavailable state and a recovery route.
- Do not register a shortcut that conflicts with a reserved OS shortcut or silently steal another app's key combination.
- Shortcuts for quick add or timer start are optional; every shortcut has a visible equivalent in MILO.

19.3 External entry points

- Start menu launch: start or restore the single process.
- Windows notification: restore process, expand and deep-link.
- File drop onto MILO: stage the dropped item, never move it.
- Startup launch: initialise in background/collapsed state according to preference.
- Update launch/restart: preserve durable state and return to a safe collapsed state.

20. Modal, menu, popover and confirmation behaviour

20.1 Layer priority

System-level handoff	Modal dialog	Popover / menu	Search overlay	Main content

Only the topmost layer receives interaction. Opening a modal must not leave a menu or popover active behind it. Closing a layer returns focus to the control that opened it where possible.

20.2 Confirmation matrix

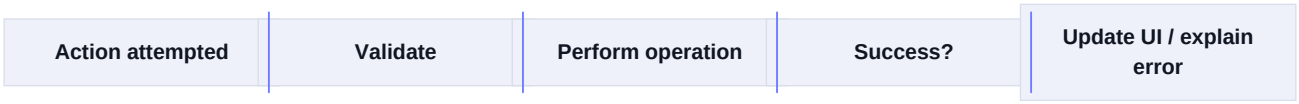
Action	Confirmation?	Confirmation copy must clarify
Collapse island	No	This hides the deck; MILO continues running.
Quit MILO	Usually no if no unsaved work; otherwise yes	MILO will stop; active background alerts may no longer be delivered if app is not resident.
Delete task	Yes	Task identity and linked data consequences.
Delete reminder/alarm	Yes	Whether future schedule/occurrences are removed.
Remove staged item	Light confirmation or clear wording	Only remove from MILO staging; original file stays on disk.
Clear all staged	Yes	Count of staged entries; original files remain untouched.
Reset data	Yes, strong confirmation	Exact data categories removed and whether backup exists.
Restore/import backup	Yes	What data will be merged/replaced and conflict policy.
Discard unsaved editor	Yes only if dirty	Unsaved changes will be lost.
Mark all complete	Yes when scope is broad	Number and filter scope of affected tasks.

20.3 Dialog button ordering

- Primary action uses a clear verb such as Save, Start, Confirm, Restore or Export.
- Secondary action uses Cancel or Back and must be safe.
- Destructive action is visually distinct and never uses ambiguous copy such as “OK”.
- Default keyboard focus should be on the safe action for destructive confirmations.
- Enter activates the primary action only when validation passes; Escape cancels unless cancellation would be unsafe.

21. Universal error, loading, empty and recovery flows

21.1 Error response pattern



Failure class	UI response	Recovery
Validation	Inline message beside field; preserve entry.	Correct field and retry.
Permission denied	Explain which capability is blocked and why it is needed.	Open Settings / Windows settings or continue without feature.
Unsupported hardware/API	Mark feature unavailable; do not display fabricated state.	Use OS settings handoff or hide unsupported action.
Database failure	Stop dependent mutation; preserve draft; log redacted diagnostic.	Retry; if persistent, offer safe recovery/backup guidance.
File missing/access denied	Mark staged item unavailable; preserve record.	Retry, reveal parent if possible, or remove from staging.
Network unavailable	Keep local features working; show offline status only for network-dependent function.	Retry when connected; no core account dependency.
Media session lost	Disable transport; retain last-known metadata as stale temporarily.	Retry session detection.
Notification failure	Keep scheduled occurrence in local store; show in-app status.	Offer OS notification settings and retry where safe.
Update check failure	Keep current version running.	Retry later; never block normal use.
Unexpected exception	Show concise error, keep shell responsive if possible.	Log diagnostic without personal content; offer restart only if necessary.

21.2 Loading rules

- Use local skeletons for data that genuinely requires retrieval; avoid full-screen spinners.
- Buttons that trigger async work enter loading and reject duplicate activation.
- Set a timeout or cancellation strategy for operations that may hang.
- Do not clear content while refreshing; keep last-known data visible with a stale indicator.
- Animation and loading feedback must not delay simple local actions.

21.3 Empty states

Section	Empty-state message intent	Primary action
Tasks	No tasks yet / no tasks match this filter.	Create task / Clear filters.
Reminders	No reminders scheduled.	Add reminder.
Timers	No saved timers; quick presets available.	Start preset / Create timer.
Alarms	No alarms configured.	Add alarm.
Files	Nothing staged. Add or drop files here; originals remain in place.	Add files.
Music	No active media session detected.	Retry detection / open supported source app help.
Notifications	You're all caught up.	Return to previous view.
Search	No results found.	Clear query / try another term.

21.4 Offline and degraded mode

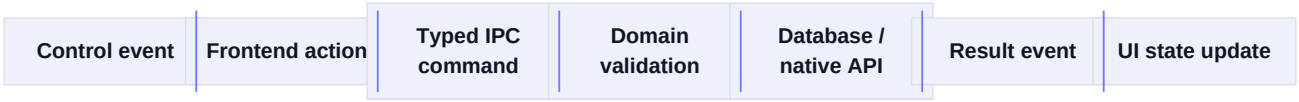
MILO's core product is local-first. A network outage must not prevent opening the island, viewing tasks, editing reminders, running timers, managing alarms, staging local file paths, or using locally available system integrations. Any feature that does depend on a remote lyrics or update service must fail independently.

22. Accessibility, keyboard and focus journeys

- All controls have accessible names, roles, states and values.
- Focus order follows visual reading order: shell header, navigation, content, contextual actions.
- On navigation, focus moves to the section heading or first meaningful control; do not leave focus on a removed element.
- After modal close, return focus to the opener.
- Keyboard: Tab/Shift+Tab moves focus; Enter/Space activates; Escape closes topmost layer; arrow keys operate sliders/selectors where standard.
- Do not use colour alone to communicate active, overdue, error, disabled or success states.
- Support Windows text scaling and high DPI; ensure controls remain usable at supported scaling factors.
- Reduced motion disables or shortens non-essential transitions; no essential information depends on animation.
- Screen-reader announcements should describe confirmed outcomes (e.g. “Timer started”, not merely “Button clicked”).

23. Data integrity and action semantics

23.1 UI-to-domain action pipeline



- Every persisted entity has a stable ID; list row identity must not depend on array index.
- Optimistic UI is allowed only when rollback is implemented and the action is safe; otherwise wait for confirmed result.
- Commands are idempotent where feasible (e.g. mark read, set preference) and guarded against double-submit.
- Use explicit domain commands such as task_create, reminder_snooze, staged_file_remove, timer_pause rather than generic unrestricted mutation.
- Do not pass raw filesystem paths or arbitrary OS commands through loosely validated frontend strings.
- Persist durable user data before reporting success.
- Never let a UI-only state update masquerade as a saved database state.

23.2 Undo strategy

Action	Undo policy
Complete task	Offer brief Undo where implementation supports safe reversal.
Dismiss notification	May offer Undo if source occurrence remains active.
Remove staged entry	Offer Undo restoring staging metadata if source still exists.
Delete task/reminder/alarm	Prefer soft-delete/undo if supported by retention model; otherwise confirmation before deletion.
Change theme/slider	Immediate reversible change; no confirmation.
Start/pause timer	Direct reversible control; no modal confirmation.
Reset data	No casual undo; require backup/explicit confirmation.

24. Canonical user journeys

Journey A — first-time user creates a task

1. Launch MILO from Start menu.
2. Welcome explains island and collapse behaviour.
3. User continues; optional preferences are offered without blocking.
4. MILO reaches collapsed state.
5. User clicks island; Overview expands.
6. User clicks Add task.
7. Task editor opens with title focused.
8. User enters title and optional due date.
9. User clicks Save.
10. MILO validates, persists task, closes editor and updates Overview summary.
11. User clicks Collapse; task remains saved.

Success criteria: task is durable after restart; no account required; the island remains resident.

Journey B — quick reminder from Overview

1. User expands island to Overview.
2. Clicks Add reminder.
3. Editor requests title, date and time; recurrence is optional.
4. User saves.
5. MILO validates schedule and persists reminder plus occurrence.
6. Confirmation shows next fire time.
7. At due time, MILO creates one notification occurrence.
8. User clicks notification; MILO restores, expands and opens reminder context.
9. User chooses Snooze; a linked snoozed occurrence is created.

Success criteria: no duplicate notification; recurrence is preserved; snooze does not overwrite recurrence.

Journey C — stage a file and upload elsewhere

1. User expands MILO and selects Files.
2. Clicks Add files or drags files into the dropzone.
3. Native file picker/drop event returns paths.
4. MILO validates metadata and stages references only.
5. User drags a staged item into a browser upload control.
6. OS drag-out passes the file to the target application.
7. MILO records outcome if observable; staged item remains available.
8. User later chooses Remove from staging.
9. Only MILO's staging record is removed; original file remains untouched.

Success criteria: original file is never moved/deleted by staging or drag-out; cancellation is safe.

Journey D — start a timer and collapse

1. User opens Timers or clicks Start timer on Overview.
2. Selects a preset or enters duration.

3. Clicks Start.
4. MILO creates timer run and begins monotonic countdown.
5. User collapses the island.
6. Timer continues in background.
7. At completion, MILO raises notification if enabled and stores completion state.
8. User opens notification; timer result view appears.
9. User dismisses or restarts.

Success criteria: timer survives navigation/collapse; completion is emitted once.

Journey E — media controls while working

1. User expands Music.
2. MILO detects an active Windows media session.
3. Metadata and confirmed playback state appear.
4. User clicks Pause.
5. MILO sends command and waits for session confirmation.
6. UI changes to Play only after confirmed paused state.
7. User switches to Tasks; media state remains monitored.
8. User returns to Music; latest session metadata is shown.

Success criteria: no fake transport state; missing session does not break other sections.

Journey F — permission denied

1. User opens a system control requiring a capability.
2. MILO explains why it needs the capability.
3. User denies or Windows reports unavailable.
4. Control returns to last confirmed state and displays a concise explanation.
5. User can continue using other sections.
6. Settings offers a route to the relevant Windows settings/help.
7. After permission is restored, user retries.

Success criteria: denial is not treated as app failure; no repeated permission nagging.

Journey G — quit and relaunch

1. User opens tray menu and chooses Quit.
2. MILO checks for unsafe unsaved work.
3. If necessary, it asks for a decision; otherwise it flushes state.
4. Background services stop and process exits.
5. User launches MILO again.
6. Single process starts, restores durable state, reconciles due schedules and opens collapsed.

Success criteria: no duplicate process, lost saved data or duplicate scheduled occurrences.

25. Complete button inventory by feature

This consolidated catalogue is intended to become the QA and implementation checklist. Repeated controls in dynamic rows must use the same behaviour contract for each entity instance.

Feature	Control	Expected result
Shell	Expand island	Open expanded shell; default Overview or deep-link target.
Shell	Collapse island	Hide deck; keep process and background services running.
Shell	Back	Return to prior view; preserve state.
Shell	Home / MILO mark	Navigate to Overview.
Shell	Search	Open search overlay and focus input.
Shell	Notifications	Open notification centre.
Shell	Settings	Open Settings and retain previous route.
Shell	Avatar	Open companion panel/customisation.
Shell	More menu	Open contextual menu; Esc/outside click dismisses.
Overview	Add task	Open task editor; Save creates; Cancel returns.
Overview	Start timer	Open quick timer; Start begins run.
Overview	Add reminder	Open reminder editor; Save schedules.
Overview	View all tasks	Navigate to Tasks.
Overview	View reminders	Navigate to Reminders.
Overview	View timers	Navigate to Timers.
Overview	Open Files	Navigate to Files.
Overview	Open Music	Navigate to Music.
Overview	Play/Pause	Control active media session; wait for confirmation.
Overview	Dismiss card	Hide summary card only; do not delete entity.
System	Volume slider	Request native volume; update from confirmed state.
System	Mute	Toggle mute; verify state.
System	Brightness slider	Set supported display brightness; verify.
System	Display selector	Select target display.
System	Wi-Fi/network	Open supported control/details or Windows Settings.
System	Bluetooth	Open supported control or Windows Settings.
System	Energy saver	Request supported OS action; verify.
System	Battery details	Expand power details.
System	Open Windows Settings	Launch relevant OS settings URI.
System	Refresh	Re-query native status.
Tasks	New task	Open blank task editor.
Tasks	Task row	Open task detail/editor.
Tasks	Complete checkbox	Persist completion; offer undo if safe.
Tasks	Edit	Open prefilled editor.
Tasks	Delete	Confirm and apply defined retention rule.
Tasks	Duplicate	Create new draft with copied fields, new identity.
Tasks	Add checklist item	Add draft checklist row.
Tasks	Remove checklist item	Remove draft/persisted child according to save state.
Tasks	Add tag	Open tag picker/create flow.

Feature	Control	Expected result
Tasks	Clear filters	Restore default list.
Tasks	Sort	Reorder list only.
Tasks	Mark all complete	Confirm count/scope, then update.
Tasks	Archive	Move out of active list if supported.
Tasks	Restore	Return archived task to active list.
Tasks	Save	Validate, persist, close editor, refresh summary.
Tasks	Cancel	Close; confirm discard only if dirty.
Reminders	Add reminder	Open reminder editor.
Reminders	Reminder row	Open details/editor.
Reminders	Date picker	Set draft date.
Reminders	Time picker	Set draft time.
Reminders	Repeat selector	Set recurrence and preview.
Reminders	Save	Persist and schedule occurrence.
Reminders	Snooze	Create linked snoozed occurrence.
Reminders	Dismiss	Acknowledge current occurrence.
Reminders	Mark complete	Complete occurrence / linked item per model.
Reminders	Delete	Confirm and remove schedule.
Reminders	Enable/disable	Pause/resume future occurrences.
Reminders	Open source task	Navigate to linked task.
Reminders	Cancel	Discard draft safely.
Timers	New timer	Open timer definition editor.
Timers	Preset	Populate duration.
Timers	Start	Create run and start countdown.
Timers	Pause	Pause run and record remaining duration.
Timers	Resume	Resume active run.
Timers	Reset	Reset after appropriate confirmation.
Timers	Stop	End run and retain history as configured.
Timers	Add time	Increase remaining duration.
Timers	Dismiss completion	Acknowledge completion.
Timers	Restart	Create new run.
Timers	Delete	Confirm; handle active run explicitly.
Timers	Save	Validate and persist timer definition.
Timers	Cancel	Discard editor draft.
Alarms	Add alarm	Open alarm editor.
Alarms	Time selector	Set local clock time.
Alarms	Repeat days	Set repeat-day pattern.
Alarms	Enable toggle	Persist enabled state and next occurrence.
Alarms	Save	Validate and persist schedule.
Alarms	Edit	Open prefilled editor.
Alarms	Delete	Confirm and remove schedule.
Alarms	Snooze	Schedule next ring after configured interval.

Feature	Control	Expected result
Alarms	Dismiss	Stop current ring; preserve recurrence.
Alarms	Alarm settings	Open sound/notification preferences.
Alarms	Cancel	Discard draft.
Files	Add files	Open native picker; stage references only.
Files	Drop files	Validate and stage dropped paths.
Files	File row	Open metadata/detail.
Files	Drag out	Begin native drag-out; preserve original.
Files	Open file	Launch default Windows application.
Files	Reveal in Explorer	Open parent folder/select file.
Files	Copy path	Copy path and show feedback.
Files	Remove from staging	Remove metadata only.
Files	Clear all staged	Confirm count; remove metadata only.
Files	Retry unavailable	Recheck path/permissions.
Files	Sort/filter	Change visible order/filter only.
Files	Privacy/help	Explain staging and retention.
Music	Play/Pause	Send media transport command; confirm.
Music	Previous	Send previous-track command.
Music	Next	Send next-track command.
Music	Seek	Seek only if supported.
Music	Volume	Adjust only clearly labelled supported volume.
Music	Show lyrics	Query supported lyrics source.
Music	Hide lyrics	Close lyrics panel.
Music	Favourite	Toggle local favourite.
Music	Open source app	Open/foreground supported source app.
Music	Retry detection	Re-enumerate media sessions.
Avatar	Avatar click	Open companion panel.
Avatar	Expression	Preview/select expression.
Avatar	Motion preference	Apply animation preference.
Avatar	Hide companion	Hide avatar; preserve feature access.
Avatar	Reset companion	Confirm and restore defaults.
Avatar	Close panel	Return to prior view.
Search	Search icon	Open overlay and focus input.
Search	Search field	Filter local results.
Search	Clear query	Clear and show suggestions/recent actions.
Search	Entity result	Open canonical entity detail.
Search	Command result	Execute selected command; confirm when needed.
Search	Close	Return to prior view and focus origin.
Notifications	Bell	Open notification centre.
Notifications	Notification row	Deep-link to source item.
Notifications	Mark read	Update read state.
Notifications	Mark all read	Clear unread state for defined scope.

Feature	Control	Expected result
Notifications	Snooze	Create snoozed occurrence.
Notifications	Dismiss	Acknowledge current alert.
Notifications	Clear history	Confirm and clear notification records only.
Notifications	Settings	Open notification settings.
Settings	Startup toggle	Register/unregister autostart; reflect confirmed state.
Settings	Start collapsed	Persist launch preference.
Settings	Theme selector	Apply selected theme.
Settings	Accent selector	Preview and persist accent.
Settings	Reduced motion	Apply accessibility preference.
Settings	Hover delay	Adjust expansion intent timing.
Settings	Auto-collapse	Apply safe inactivity policy.
Settings	Full-screen suppression	Suppress automatic expansion over full-screen apps.
Settings	Notification help	Open OS notification settings/help.
Settings	Test notification	Send test alert.
Settings	Shortcut recorder	Capture and validate shortcut.
Settings	Reset shortcut	Restore default shortcut.
Settings	Timer preset editor	Manage presets.
Settings	Default snooze	Set future default interval.
Settings	Clear staged	Open staging-clear confirmation.
Settings	Export data	Choose destination/scope; create export.
Settings	Import/restore	Validate backup, preview and confirm.
Settings	Reset data	Strong confirmation; reset only stated scope.
Settings	Check updates	Query update source; display result.
Settings	Release notes	Open in-app notes or trusted page.
Settings	Diagnostics	Offer local export; no automatic transmission.
Settings	About/licenses	Show version and notices.
Settings	Back	Return to exact prior route.

26. Implementation mapping for Codex

26.1 Suggested frontend structure

The exact repository layout is governed by the Technical Requirements Document. The following conceptual separation is required even if file names differ.

Layer	Responsibility	Must not contain
App shell	Lifecycle presentation, collapsed/expanded state, global overlays.	Database access or direct OS calls.
Router/navigation	Route stack, selected section, deep-link resolution, focus restoration.	Feature-specific persistence logic.
Feature views	Rendering lists, forms, detail views, control states.	Raw SQL or direct native API calls.
Domain services	Validation, entity rules, scheduling semantics, orchestration.	React component concerns.
IPC client/contracts	Typed request/response DTOs, error mapping.	Unvalidated arbitrary commands.
Rust/native adapters	Windows APIs, filesystem, tray, media, notifications, window positioning.	Presentation state or UI copy.
Persistence layer	SQLite queries, migrations, transactions, repositories.	Direct UI event handling.

26.2 Action contract

Every button handler should follow a consistent pattern: identify the command; validate required local input; set a pending state; invoke a typed domain action; receive a typed success/error result; update canonical state; announce the outcome; restore focus where needed. Async handlers must guard against double-submit and stale component references.

26.3 Required event/result shape

Concept	Required fields (illustrative)
Action request	request_id, command, entity_id (if applicable), payload, issued_at.
Action success	request_id, entity_id, canonical_record or updated state, completed_at.
Action error	request_id, stable error_code, safe user_message, retryable, field_errors (if any).
Deep link	target_section, entity_type, entity_id, origin_context.
Notification action	notification_id, occurrence_id, action_type, entity_id.

26.4 Suggested component/action naming

- Use stable IDs in this document (e.g. TSK-03) as test references and optionally code comments/test names.
- Use domain verbs: createTask, completeTask, snoozeReminder, startTimer, stageFiles, removeStagedFile, setSystemVolume.
- Keep presentation handlers thin; centralise validation and state transitions.
- All native calls must have typed return values and explicit unsupported/permission/error outcomes.
- Use a single source of truth for entity state and derive overview counts from canonical records.

27. QA acceptance matrix and test scenarios

Test ID	Scenario	Pass condition
FLOW-001	Fresh install → first launch	Welcome completes; database/profile initialised; collapsed island shown.
FLOW-002	Second launch while running	Existing instance restored; no duplicate process/window.
FLOW-003	Click island	Expanded shell opens and focus is usable.
FLOW-004	Hover then leave before delay	No expansion.
FLOW-005	Collapse with active timer	Island collapses; timer continues.
FLOW-006	Navigate between sections	No draft loss, duplicate routes or scroll reset.
FLOW-007	Task save	Task persists and appears in list/Overview.
FLOW-008	Task cancel dirty	Discard confirmation; task unchanged if discarded.
FLOW-009	Task completion	Completion persists and overdue state clears.
FLOW-010	Reminder due	One occurrence and at most one notification.
FLOW-011	Snooze reminder	New snoozed occurrence; recurrence preserved.
FLOW-012	Timer sleep/resume	Remaining time reconciles correctly; no duplicate completion.
FLOW-013	Alarm repeat	Correct next local occurrence; no duplicate ring.
FLOW-014	File picker stage	Original path unchanged; metadata persisted.
FLOW-015	Drag file to target	Target receives file; MILO does not move/delete original.
FLOW-016	Cancel drag-out	Staged item remains available.
FLOW-017	Remove from staging	Only staging record removed.
FLOW-018	No media session	Music idle state; other sections remain operational.
FLOW-019	Media command failure	UI returns to confirmed state with error.
FLOW-020	Permission denied	Feature degrades gracefully; other app functionality works.
FLOW-021	Search deep link	Correct entity opens; Back restores query/results.
FLOW-022	Notification deep link	Existing process restores and opens target item.
FLOW-023	Settings theme	Theme applies immediately and persists.
FLOW-024	Shortcut conflict	Registration failure visible with remediation.
FLOW-025	Quit/relaunch	Durable state restored; no duplicate schedules.
FLOW-026	DPI/display change	Island repositions and remains usable.
FLOW-027	Reduced motion	Non-essential motion is reduced/removed.
FLOW-028	Keyboard-only journey	All controls reachable, operable and focus returns correctly.
FLOW-029	Database write failure	Draft preserved; no false success.
FLOW-030	Offline mode	Core local workflows continue.

27.1 Definition of done for an interaction

- Control is visible only when the action is supported and relevant.
- Control has a stable ID, accessible name, tooltip where icon-only, and keyboard operation.
- Activation performs the documented action exactly once.
- Pending, success, disabled, error and cancellation states are designed and implemented.
- Persistence/native operation is confirmed before the UI claims success.
- Navigation and focus behaviour are correct after completion/cancellation.
- Unit/integration/UI tests cover the happy path and at least one failure/cancellation path.

- Analytics/diagnostics, if any, do not capture private content or file contents.

28. Decisions that must not be silently invented

The following details require explicit product/technical decisions if not already fixed in the companion documents. Codex should record a proposed default and ask rather than inventing consequential behaviour.

Decision	Why it matters	Safe implementation stance until resolved
Completing task with linked reminder	Could leave a reminder firing for a completed task.	Do not silently cancel or retain; implement an explicit, testable rule after decision.
Alarm wake-from-sleep guarantee	Windows sleep and app lifecycle affect delivery.	Do not promise wake-from-sleep; document actual tested limits.
Timer behaviour when Windows sleeps	Elapsed-time semantics may vary by user expectation.	Use reliable elapsed-time reconciliation; document pause/continue semantics.
Task deletion model	Hard vs soft delete affects linked reminders/history.	Follow database retention rules; confirm destructive operation.
Staging retention duration	Affects privacy and stale file paths.	Keep user-controlled staging until explicit removal; do not delete source.
Auto-collapse timeout	Can interrupt user work if aggressive.	Default conservative; suppress during typing, modal, drag and focus interactions.
Lyrics provider/licensing	Source availability and rights vary.	Only show lyrics from approved supported source; otherwise unavailable.
System toggle support	Windows API availability differs by hardware/version.	Use verified APIs or settings handoff; never fake control.
Default global shortcut	Can conflict with user/system shortcuts.	Make configurable and validate registration; provide visible fallback.
Notification sound defaults	Can be disruptive/accessibility-sensitive.	Use restrained defaults and explicit settings.

28.1 Product rules that are already firm

- MILO is one Windows desktop application, not a website.
- The floating island is the primary shell and expands downward from the top edge.
- The app remains resident when collapsed; collapse is not quit.
- Core features are local-first and do not require an account.
- Staged files are references/temporary holding entries; staging and drag-out must not silently move or delete originals.
- Media and lyrics depend on supported integrations; no fake metadata or fabricated lyrics.
- The avatar is optional/subtle and never blocks the functional UI.

29. External technical references

The following official references were consulted to ground the interaction model in the current Tauri v2 capability surface. They are implementation references, not a substitute for validating Windows-specific APIs on the target Windows versions and hardware.

Reference	URL	Relevance
Tauri v2 Features & Recipes	https://v2.tauri.app/plugin/	Official overview of plugins such as global shortcuts, notifications, filesystem, opener, SQL, updater and autostart.
Tauri v2 Window API	https://v2.tauri.app/reference/javascript/api/namespacewindow/	Window lifecycle and events, including drag/drop event handling and scale changes.
Tauri v2 Configuration	https://v2.tauri.app/reference/config/	Window configuration, tray icon configuration and platform-specific configuration.
Tauri v2 Plugin Development	https://v2.tauri.app/develop/plugins/	Lifecycle hooks and Rust/JavaScript plugin boundaries.

29.1 Verification note

The references above were located during preparation of this specification. Codex should verify exact API versions, plugin support tables, permissions/capabilities and Windows-specific behaviour against the installed Tauri version and the supported Windows baseline before implementation. System controls such as brightness, Bluetooth, Wi-Fi and power mode require particular care because hardware, drivers and Windows releases differ.

30. Master instruction for Codex

Treat this document as the authoritative user-flow and interaction contract for MILO v1.0. Do not implement a button, gesture, route or notification action without mapping it to the documented behaviour and a test.

- Read all six MILO documents before changing architecture or building features.
- First produce a route/state inventory and map each documented control ID to a planned component, handler, domain command and test.
- Flag missing product decisions from Section 28 before implementing behaviour that affects data loss, scheduling guarantees, file safety or system settings.
- Build the shell lifecycle and navigation model first. Then implement feature vertical slices in the order defined by the Implementation Plan.
- Use the same domain action for a feature regardless of entry point: Overview, section screen, search, tray, shortcut or notification.
- Implement every async control with pending, success and error states; preserve drafts on failure.
- Never claim an OS/native action succeeded until the native layer confirms it.
- Never delete/move source files as part of staging or drag-out.
- Never fabricate media state, lyrics, battery values, connectivity state or unsupported system controls.
- Keep MILO usable when optional integrations fail.
- Add automated tests for every control ID and journey where practical; at minimum cover all critical/destructive/scheduling/file interactions.
- Update this specification when an approved product decision changes a route, action result or confirmation rule.

Final implementation principle

MILO should feel effortless because the complexity is handled consistently underneath. The user should never have to wonder whether a button worked, whether a timer kept running, whether a file was moved, whether a reminder was duplicated, or how to get back to where they were. Every interaction should have a clear beginning, a confirmed outcome, and a safe next step.